
MENU-DRIVEN STRING OPERATIONS USING C PROGRAMMING

Name : Rudra Pratap Singh
SAP ID : 590032811
Batch : 16

1. Problem Statement

Write a menu-driven C program to perform the following operations using separate functions:

- Find the longest word in a sentence.
- Check if one string is a rotation of another.

2. Aim

To design and implement a menu-driven program in C in which each operation is written as a separate, reusable function, and the user can repeatedly choose an operation until he or she decides to exit.

3. Concepts Used

- **Strings:** character arrays terminated by the null character `'\0'`.
- **Functions:** modular programming, function prototypes, passing arrays to functions, and return values.
- **Menu-driven control:** a `do-while` loop combined with a `switch` statement.
- **Library functions:** `strlen()`, `strcpy()`, `strcat()`, `strstr()`, `strcspn()` from `<string.h>` and `isalnum()` from `<ctype.h>`.
- **Safe input:** `fgets()` is used instead of `gets()` so that sentences containing spaces can be read without risking buffer overflow.

4. Functions Used in the Program

Function	Return Type	Purpose
<code>main()</code>	<code>int</code>	Displays the menu, accepts the user's choice and calls the required function using switch-case. Repeats until Exit is chosen.
<code>displayMenu()</code>	<code>void</code>	Prints the list of available operations.
<code>readLine()</code>	<code>void</code>	Reads a complete line (with spaces) using <code>fgets()</code> and removes the trailing newline character.
<code>longestWord()</code>	<code>void</code>	Scans the sentence, finds the longest word and prints it with its length.
<code>isRotation()</code>	<code>int</code>	Returns 1 if the second string is a rotation of the first string, otherwise returns 0.

5. Algorithms

5.1 Longest Word in a Sentence

1. Read the sentence from the user.
2. Set `longest = ""`, `maxLen = 0` and `curLen = 0`.

3. Traverse the sentence character by character up to and including the null character.
4. If the character is a letter or digit, append it to the current word and increase curLen.
5. Otherwise (space, punctuation or end of string) the current word has ended. If $\text{curLen} > \text{maxLen}$, copy the current word into longest and update maxLen. Then reset curLen to 0.
6. Stop when the null character is reached.
7. If maxLen is 0, print "No word found"; otherwise print longest and maxLen.

5.2 Rotation of a String

A string **S2** is a rotation of string **S1** if it can be obtained by moving some characters from the front of **S1** to its end. For example, "cdab" is a rotation of "abcd". The key idea is that every rotation of **S1** always appears as a substring of **S1 + S1**:

$S1 = \text{abcd} \rightarrow S1 + S1 = \text{abcdabcd} \rightarrow \text{contains "cdab", "bcda", "dabc", "abcd"}$

1. Read the two strings **S1** and **S2**.
2. If the lengths of **S1** and **S2** are not equal, **S2** cannot be a rotation of **S1**; return 0.
3. Create a temporary string **temp = S1 + S1** using `strcpy()` and `strcat()`.
4. If **S2** is found inside **temp** (using `strstr()`), return 1; otherwise return 0.
5. Print the appropriate message in `main()` based on the returned value.

6. Source Code

The complete program is given below (file name: `string_operations.c`).

```

/*
 * Program : Menu-driven String Operations
 * Author   : Rudra Pratap Singh
 * SAP ID   : 590032811
 * Batch    : 16
 *
 * Operations:
 * 1. Find the longest word in a sentence
 * 2. Check whether one string is a rotation of another
 */

#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 200

/* Function prototypes */
void displayMenu(void);
void readLine(char str[], int size);
void longestWord(char sentence[]);
int isRotation(char str1[], char str2[]);

/* ----- main ----- */
int main()
{
    int choice;
    char sentence[MAX], str1[MAX], str2[MAX];

```

```

do
{
    displayMenu();
    printf("Enter your choice: ");

    if (scanf("%d", &choice) != 1)
    {
        /* invalid (non-numeric) input: clear the buffer */
        while (getchar() != '\n');
        printf("\nInvalid input! Please enter a number (1-3).\n");
        choice = 0;
        continue;
    }
    while (getchar() != '\n'); /* remove leftover newline */

    switch (choice)
    {
        case 1:
            printf("\nEnter a sentence: ");
            readLine(sentence, MAX);
            longestWord(sentence);
            break;

        case 2:
            printf("\nEnter first string : ");
            readLine(str1, MAX);
            printf("Enter second string : ");
            readLine(str2, MAX);

            if (isRotation(str1, str2))
                printf("\nResult: \"%s\" IS a rotation of \"%s\".\n",
                    str2, str1);
            else
                printf("\nResult: \"%s\" is NOT a rotation of \"%s\".\n",
                    str2, str1);
            break;

        case 3:
            printf("\nThank you for using the program. Goodbye!\n");
            break;

        default:
            printf("\nInvalid choice! Please select 1, 2 or 3.\n");
    }
} while (choice != 3);

return 0;
}

/* ----- Display the menu ----- */
void displayMenu(void)
{
    printf("\n===== \n");
    printf("          STRING OPERATIONS MENU\n");
    printf("===== \n");
    printf(" 1. Find the longest word in a sentence\n");
    printf(" 2. Check if one string is a rotation of another\n");
    printf(" 3. Exit\n");
    printf("----- \n");
}

/* ----- Read a full line safely and remove the newline ----- */
void readLine(char str[], int size)
{
    if (fgets(str, size, stdin) != NULL)
        str[strcspn(str, "\n")] = '\0';
    else
        str[0] = '\0';
}

```

```

}

/* ----- Operation 1: find the longest word ----- */
void longestWord(char sentence[])
{
    char longest[MAX] = "";
    char current[MAX];
    int  maxLen = 0, curLen = 0, i;

    for (i = 0; ; i++)
    {
        /* a word consists of letters and digits only */
        if (isalnum((unsigned char)sentence[i]))
        {
            current[curLen++] = sentence[i];
        }
        else
        {
            /* space, punctuation or end of string ends the current word */
            if (curLen > maxLen)
            {
                current[curLen] = '\0';
                strcpy(longest, current);
                maxLen = curLen;
            }
            curLen = 0;

            if (sentence[i] == '\0')
                break;
        }
    }

    if (maxLen == 0)
        printf("\nNo word found in the sentence.\n");
    else
        printf("\nLongest word : %s\nLength      : %d characters\n",
               longest, maxLen);
}

/* ----- Operation 2: check rotation ----- */
/* Returns 1 if str2 is a rotation of str1, otherwise 0.
   Idea: if str2 is a rotation of str1, then str2 is a
   substring of str1 + str1. */
int isRotation(char str1[], char str2[])
{
    char temp[2 * MAX];

    if (strlen(str1) != strlen(str2))        /* different lengths: impossible */
        return 0;

    strcpy(temp, str1);
    strcat(temp, str1);                      /* temp = str1 + str1 */

    if (strstr(temp, str2) != NULL)
        return 1;

    return 0;
}

```

7. Sample Output

Text typed by the user is shown in bold blue.

Output 1: Finding the longest word

```
=====
          STRING OPERATIONS MENU
=====
  1. Find the longest word in a sentence
  2. Check if one string is a rotation of another
  3. Exit
-----
Enter your choice: 1

Enter a sentence: I love programming in C language

Longest word : programming
Length       : 11 characters
```

Output 2: Checking rotation – positive case

```
=====
          STRING OPERATIONS MENU
=====
  1. Find the longest word in a sentence
  2. Check if one string is a rotation of another
  3. Exit
-----
Enter your choice: 2

Enter first string : abcd
Enter second string : cdab

Result: "cdab" IS a rotation of "abcd".
```

Output 3: Checking rotation – negative case

```
=====
          STRING OPERATIONS MENU
=====
  1. Find the longest word in a sentence
  2. Check if one string is a rotation of another
  3. Exit
-----
Enter your choice: 2

Enter first string : abcd
Enter second string : acbd

Result: "acbd" is NOT a rotation of "abcd".
```

Output 4: Invalid choice and Exit

```
=====
      STRING OPERATIONS MENU
=====
  1. Find the longest word in a sentence
  2. Check if one string is a rotation of another
  3. Exit
-----
Enter your choice: 7

Invalid choice! Please select 1, 2 or 3.

=====
      STRING OPERATIONS MENU
=====
  1. Find the longest word in a sentence
  2. Check if one string is a rotation of another
  3. Exit
-----
Enter your choice: 3

Thank you for using the program. Goodbye!
```

8. Test Cases

No.	Operation	Input	Output
1	Longest word	I love programming in C language	programming (11)
2	Longest word	Hello, world!	Hello (5)
3	Longest word	(only spaces)	No word found
4	Rotation	abcd , cdab	Is a rotation
5	Rotation	waterbottle , erbottlewat	Is a rotation
6	Rotation	abcd , acbd	Not a rotation
7	Rotation	abc , abcd (different length)	Not a rotation
8	Menu	7 (invalid number)	Invalid choice message
9	Menu	xyz (non-numeric)	Invalid input message

9. Conclusion

The program successfully performs both string operations through a simple menu. Every task is written as a separate function, which keeps main() short, makes the code easy to read and test, and allows the functions to be reused in other programs. The longest word is found in a single pass over the sentence, and the rotation check is done with the S1 + S1 technique after a quick length comparison. The program also handles invalid menu input, punctuation in sentences and empty input without crashing. Through this project, the concepts of strings, functions, loops, switch-case and the standard string library were practised.